

G H Raison College of Engineering and Management, Pune

(An Empowered Autonomous Institute affiliated to Savitribai Phule Pune University, NAAC Accredited "A+" Grade)

Department of Computer Science and Engineering (Data Science)

TAE 1 (20 Marks) – Database Management Systems

Business Problem, ER/EER Diagram, Relational Schema & Normalization Plan

23UDSPCL3508 / 23UDSPCP3508 : Database Management Systems

Gym Membership Management Database

Student Name	Siddharam Achaler
Roll Number	P57
PRN	24ACDS1121072
Section	P
Class	T.Y. B.Tech CSE (Data Science)
Submission Date	28 July 2026

1. Problem Statement and System Requirements

1.1 Business Domain

For this project I have chosen a domain that most people around me actually deal with but that colleges rarely pick as an example - a neighbourhood gym / fitness centre. Almost every local gym I have seen (including the one near my hostel) still runs its membership on a physical register or, at best, a WhatsApp broadcast list and a plain Excel sheet maintained by the receptionist. There is no proper system to quickly check which member's plan is about to expire, whether a particular member has actually been coming regularly, or how much fee is still pending from a member for the month.

The system I am proposing, the Gym Membership Management Database (GMMD), is meant to bring all of this into one place - the members who join the gym, the trainers assigned to guide them, the membership plans available, the day-to-day attendance of each member, and the fee payments made against each membership. Once this is stored in a proper relational database, things like "list all members whose plan expires this week" or "total revenue collected this month" become a simple SELECT query instead of someone manually going through a register.

1.2 Entities in the System

- Trainer – gym staff member who is assigned to guide one or more members.
- MembershipPlan – the different plans the gym offers (e.g. Monthly, Quarterly, Annual) along with their fee and duration.
- Member – a person who has joined the gym under a particular plan, optionally assigned to a trainer.
- Attendance – a record of a member checking in to the gym on a given date.
- Payment – a record of a fee payment made by a member towards their membership.

1.3 Constraints and Business Rules

- Every member must be enrolled in exactly one membership plan (PlanID cannot be null in Member).
- A member may or may not have a trainer assigned yet, so TrainerID in Member is allowed to be null, but if it is filled it must reference an existing Trainer.
- A member can mark attendance at most once per calendar date - enforced through a composite PRIMARY KEY on (MemberID, AttendanceDate).

- The AmountPaid in Payment can never be negative or zero - a CHECK constraint handles this.
- A member's very first attendance date should never be earlier than their JoinDate. A plain CHECK constraint cannot compare a column of Attendance against a column sitting in a different table (Member), so this rule will be enforced through a trigger, the way cross-table validation rules are usually handled at the trigger level in DBMS.
- Age of a member should be at least 14 years - CHECK constraint on Member.Age.
- A member, trainer or plan cannot be deleted if it is still referenced by existing attendance or payment records (ON DELETE RESTRICT), so that old billing/attendance history is never silently lost. Only the Member → Trainer link is allowed to go to NULL if a trainer leaves the gym (ON DELETE SET NULL), since attendance and payment history should still be traceable even without a linked trainer record.

1.4 Main Transactions

Transaction	What it does
Register New Member	Receptionist adds a new member along with the plan chosen and, if available, the trainer assigned.
Assign / Change Trainer	An existing member is linked to a trainer, or the trainer is changed if the member switches.
Mark Daily Attendance	Front-desk marks a member present for the day; duplicate entries for the same date are blocked by the key.
Record Fee Payment	A payment made by the member is logged against their membership with date and amount.
Generate Expiry / Revenue Report	A reporting query that lists members whose plan is close to expiry, or totals the fee collected in a period.

2. Reference Material Used

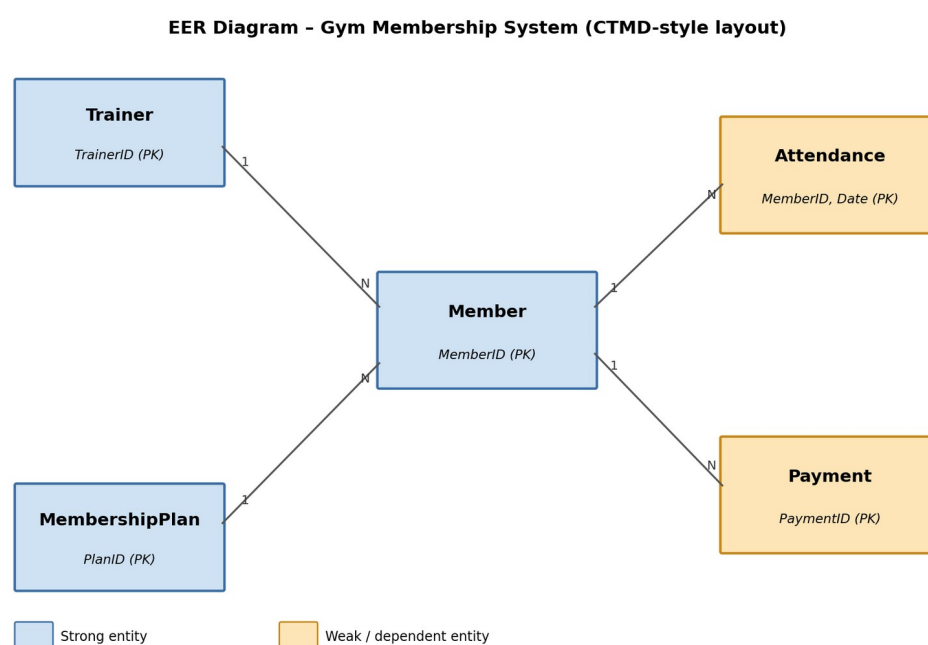
Since this project is not based on a government dataset, the references here are fairly simple and practical. I looked at how the check-in flow works in gym apps like Cult.fit and the membership tracking used by Gold's Gym franchises - both separate a member's basic profile from their day-to-day check-ins and from their payment history, instead of cramming everything into one sheet. I have followed the same separation in Section 4 by keeping Member, Attendance and Payment as three distinct tables rather than one wide table.

I also spoke to the person who manages the register at the gym I go to, mainly to understand what columns a real register actually has (name, contact number, plan, trainer, and a payment date noted at the back of the page whenever

someone pays) - that register format is what I have used as the starting point for the raw/unnormalised data in Section 4.1.

3. Conceptual Schema Design (EER Diagram)

The diagram below has five entities. Trainer, MembershipPlan and Member are shown as strong (independent) entities in blue because each of them can exist on its own without depending on another entity's existence. Attendance and Payment are shown in amber because they only make sense once a Member already exists - an Attendance row cannot exist without a Member checking in, and a Payment row cannot exist without a Member making that payment. Every relationship here turned out to be a simple 1:N, so, like in similar small-scale systems, there was no need for any M:N relationship or a bridge/junction entity.



3.1 Keys and Cardinality Summary

Entity	Primary Key	Foreign Keys	Participates In
Trainer	TrainerID	-	1:N with Member
MembershipPlan	PlanID	-	1:N with Member
Member	MemberID	PlanID, TrainerID	1:N with Attendance, 1:N with Payment
Attendance	MemberID + AttendanceDate	MemberID	weak entity, depends on Member
Payment	PaymentID	MemberID	depends on Member

4. Relational Schema and Normalisation Plan

4.1 Raw / Unnormalised Data

Instead of using a ready-made dataset, I built a raw sheet that looks like what the gym's front-desk register actually has - one row per member, with all of that member's payment dates and amounts squeezed into a single column, separated by semicolons, exactly the way it is written by hand at the back of the register page.

MemberID	MemberName	ContactNo	PlanName	PlanFee	TrainerName	PaymentRecords (multivalued)
G001	Rohit Kadam	9822011223	Quarterly	4500	Sanjay More	05-01-2026:1500; 05-02-2026:1500
G002	Neha Deshpande	9822033445	Monthly	1800	Sanjay More	10-01-2026:1800
G003	Aditya Kulkarni	9822055667	Annual	15000	-	02-01-2026:15000

(This is a sample of 3 rows out of the full register of about 35 active members I used while designing this schema.)

4.2 Step 1 - First Normal Form (1NF)

Problem: the PaymentRecords column has more than one value packed into a single cell (one payment date and amount, then a semicolon, then the next). This is a repeating group, and it stops me from asking something as basic as "how much has Rohit Kadam paid in total" without first manually splitting this text.

Fix - pull the repeating group into its own table, one row per (member, payment):

MemberExport_1NF (MemberID, MemberName, ContactNo, PlanName, PlanFee, TrainerName)

MemberPaymentRaw_1NF (MemberID, PlanName, PaymentDate, AmountPaid)

4.3 Step 2 - Second Normal Form (2NF)

Problem: the key of MemberPaymentRaw_1NF is the pair (MemberID, PaymentDate). But I have also kept PlanName in this table, and PlanName only depends on MemberID - it has nothing to do with which PaymentDate is being looked at. That is a partial dependency, because PlanName does not need the full (MemberID, PaymentDate) key to be determined.

FD found: MemberID → PlanName (partial - does not involve PaymentDate)

Fix - move plan info out of the payment table:

MemberPlan_2NF (MemberID, PlanName)

MemberPayment_2NF (MemberID, PaymentDate, AmountPaid)

4.4 Step 3 - Third Normal Form (3NF) / BCNF

Problem: in MemberExport_1NF, PlanFee depends on PlanName and not directly on MemberID - if "Quarterly" plan is used by 20 members, its fee of 4500 is repeated 20 times, and if the gym revises the quarterly fee I would have to update every single one of those rows (a classic update anomaly). The same problem shows up with TrainerName - if I later add a TrainerSpecialization column, it would depend on TrainerName and not on MemberID.

FDs found: MemberID → MemberName, ContactNo, PlanName, TrainerName

PlanName → PlanFee (transitive)

Fix - pull MembershipPlan and Trainer out as their own tables keyed on a surrogate ID, and reference them by foreign key from Member instead of repeating their text values everywhere. Once this is done, every non-key column depends on the whole key and nothing but the key, so the design reaches 3NF - and because every determinant here is also a key of its own table, it satisfies BCNF as well.

4.5 Final Normalised Schema (3NF/BCNF)

```
Trainer (  
    TrainerID INT PRIMARY KEY,  
    TrainerName VARCHAR(80) NOT NULL,  
    Specialization VARCHAR(50),  
    ContactNo VARCHAR(15)  
)
```

```
MembershipPlan (  
    PlanID INT PRIMARY KEY,  
    PlanName VARCHAR(40) NOT NULL UNIQUE,  
    DurationMonths INT CHECK (DurationMonths > 0),  
    Fee DECIMAL(8,2) CHECK (Fee > 0)  
)
```

```
Member (  
    MemberID INT PRIMARY KEY,  
    FullName VARCHAR(80) NOT NULL,  
    ContactNo VARCHAR(15) NOT NULL,  
    Age INT CHECK (Age >= 14),  
    JoinDate DATE NOT NULL,  
    PlanID INT NOT NULL REFERENCES MembershipPlan(PlanID) ON  
DELETE RESTRICT,  
    TrainerID INT REFERENCES Trainer(TrainerID) ON DELETE SET  
NULL  
)
```

```
Attendance (  

```

```

    MemberID INT NOT NULL REFERENCES Member(MemberID) ON DELETE
    RESTRICT,
    AttendanceDate DATE NOT NULL,
    CheckInTime TIME,
    PRIMARY KEY (MemberID, AttendanceDate)
)

Payment (
    PaymentID INT PRIMARY KEY,
    MemberID INT NOT NULL REFERENCES Member(MemberID) ON DELETE
    RESTRICT,
    PaymentDate DATE NOT NULL,
    AmountPaid DECIMAL(8,2) CHECK (AmountPaid > 0),
    PaymentMode VARCHAR(20) CHECK (PaymentMode IN
('Cash', 'UPI', 'Card'))
)

```

4.6 Integrity Constraints Summary

Constraint Type	Where Applied
Entity Integrity	Every table has its own single-column surrogate primary key (TrainerID, PlanID, MemberID, PaymentID) which is non-null and unique; Attendance uses a composite key (MemberID, AttendanceDate) instead since no single natural surrogate was needed for it.
Referential Integrity	Foreign keys (PlanID and TrainerID in Member, MemberID in Attendance and Payment) all reference existing rows; ON DELETE RESTRICT is used for Plan/Attendance/Payment links so billing history is never silently broken, while ON DELETE SET NULL is used only for the Member → Trainer link.
Domain Constraints	CHECK constraints on Age (≥ 14), Fee/AmountPaid (> 0), DurationMonths (> 0), and PaymentMode (restricted to Cash/UPI/Card).
Candidate Key Constraints	UNIQUE on PlanName, and composite PRIMARY KEY (MemberID, AttendanceDate) in Attendance.

5. Target RDBMS Software

I am planning to implement this schema in MySQL 8.0 using MySQL Workbench, mainly because I am already comfortable with Workbench's ER-to-schema tools from our lab sessions, and because every CHECK constraint used above (Age ≥ 14 , Fee > 0 , AmountPaid > 0) is a simple boolean expression, which MySQL has supported properly since version 8.0.16. The composite primary key on Attendance and the mixed ON DELETE RESTRICT / SET NULL foreign key behaviour used in this design are also both fully supported in MySQL 8. All the DDL, DML, joins, subqueries and the trigger for the join-date validation rule from Section 1.3 will be written against this same schema in the next phase, without changing any of the entities or business rules defined here.

Submitted by: Siddharam Achaler (Roll No. P57, PRN 24ACDS1121072, Section P)

T.Y. B.Tech CSE (Data Science), G H Raisonni College of Engineering and Management, Pune